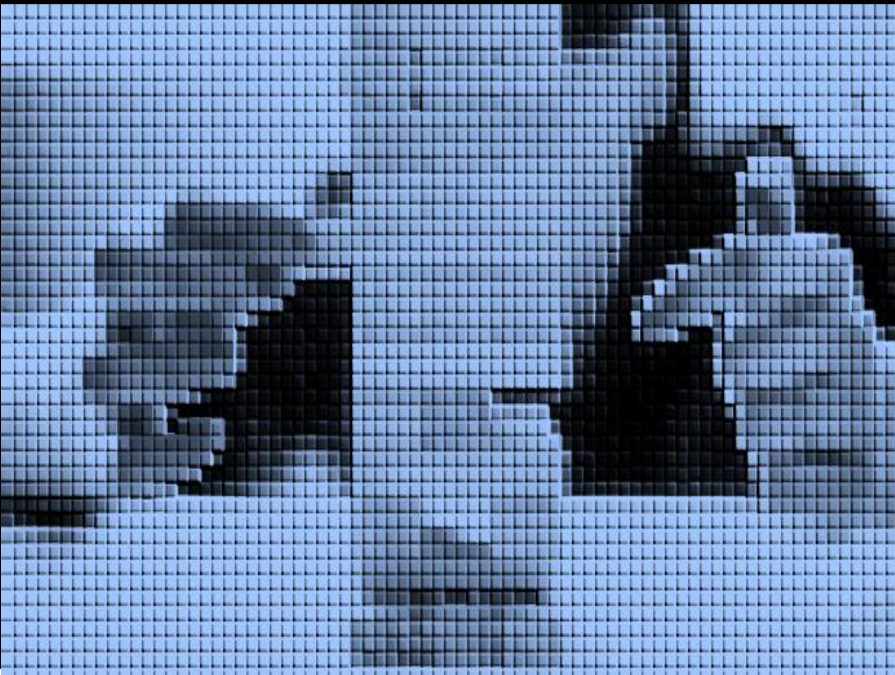


COMS10016: Imperative and Functional Programming



Marking Guide

Tilo Burghardt

Marks at University

- In schools, there is a common marking scale used where "if you get everything right, you get full marks".
- The scale used in universities isn't like that at all.
- At university, each mark is usually presented out of 100.
- The idea is that marks from 0 to 100 represent the entire range of human ability and achievement, this scale is also non-linear.
- That means: the first few marks are easy to get, and higher marks are harder and harder to get.
- A mark somewhere between 50 – 65 represents an average Bristol University student's mark.
- Since 100 represents a level of greatest genius, marks of 70+ are tough to get, 80+ are very rare, and marks of 100 are vanishingly rare.

General Marking Bands

- University staff are very familiar with this marking scale, through subject conventions, experience, and a system of external examiners.

A rough Rule of Thumb is:

- 40: passable
- 50: good
- 60: very good
- 70: excellent, above and beyond what is expected
- 80: mastery of the subject area
- 90: novel and publishable in a research journal

Degree Grades

At the end of a BSc degree, you will get a grade, based on an average mark out of 100; the grades are at first approximation:

- "first class" (1) for 70 or more
 - "upper second class" (2.1) for 60 to 69
 - "lower second class" (2.2) for 50 to 59
 - "third class" (3) for 40 to 49
-
- There are various lesser awards and credit transfers.
 - Masters degrees use the same scale, except with a 50% pass mark, so getting <50% in an MEng may result in a BSc.
 - Full details on the subject can be found at the 'Regulations and Code of Practice for Taught Programmes' on the University of Bristol websites

COMS10016 Marking

- Getting the foundations for a career in computing in an age of AI is about understanding the concepts behind programs and the language used to communicate about them with people and machines – initially we focus on specs, tests, correctness, vocabulary around precise concepts, and how computations operate, later on high efficiency, professional design, and creativity.
- This first course is about the concepts that underpin programming, GenAI utilisation to just write code will be introduced in Software Tools later in the year and will form core parts of year two units like Computer Systems A or the Software Engineering Project.
- Programs first need to be completely correct {whether you write them or review AI written ones} and in target style - aim at this first!
- Our labs, formative coursework, worksheets and exercises will help you incrementally to expand your skill set over the term. There will be a must-pass exam (you will not pass the unit without passing the exam) on at the end of the unit worth 80% of the unit mark, and two small pieces of summative coursework worth 10% of the unit mark each which are there to learn practically, get ready for the exam and pick up marks towards your unit score on the way.
- Each coursework also prepares you for the exam and is deliberately split into two halves: a first half (worth the first 50% or so of the mark) which is a totally closed task, and a second half which is an open-ended task. The unit marking scheme aims at making your total unit mark match the university marking scale.

Imperative Coursework – Closed Part

- The closed part is about precision, specifications and test-driven development; every detail is provided, so you know exactly what to try to achieve.
- You are also given in advance a complete set of tests which will be used in marking to measure success.
- That means you have every chance to get full marks for this closed part and, failing that, you will know exactly what mark you are going to get when you submit.
- That means:
 - (a) we *want everybody* to get full marks in this part, and
 - (b) we can objectively automate parts of the marking.

Imperative Coursework – Open-ended Part

- For the open part, you will only be given a coarse task with a clear objective for the implementation, yet with almost no guidance.
- This means you will be free to go in any direction you want within the bounds of the task and topic. Note that all programs must effectively and efficiently produce practical results.
- You are expected to use the same skills as in the first half, but also create your own tests (which your program should of course pass).
- You must always submit compiling code only, but it is perfectly acceptable to stop when you run out of time.
- The marking will be done by hand, based on how much practical task success, advanced ability, knowledge, skill, creativity and effort you demonstrate.

Aims, Workload and Time

- You should keep track of whether or not you are exceeding approximately 10h per week on your lab and coursework for the entire unit (excluding lectures, recap and further reading), and only do more if you are able and willing.
- Everyone should aim to finish at least all functional worksheets and implement the closed parts of the imperative assignments (both formative and summative).
- Program for at least for about 10 hours every week! Always concentrate on making your programming skills more efficient, engage in our extra exercises and actively ask us during labs. Attend all learning activities throughout.

Don't tackle the open part at all, unless you are sure that you have the closed part sorted.

Coursework Marking Guideline (40)

PASSABLE SUBMISSION Mark (40)

- + correctly submitted and compiling code;
- + 80% of the provided tests for the closed task pass;
- + full compliance with the provided tests/function naming;

The submitted material usually shows most of the following:

- + programming style and design only partly adhere to the taught concepts and guidelines;
- + widely correct use of basic techniques;
- + demonstration of basic coding competence;

Coursework Marking Guideline (50)

GOOD SUBMISSION Mark (50)

all criteria of a passable submission plus:

+ 100% of the provided tests for the closed task pass;

The submitted material usually shows most of the following:

- + programming style and design mostly adhere to the taught concepts and guidelines;
- + effective use of basic techniques;
- + demonstration of good coding competence;
- + evidence of generally appropriate use of techniques;
- + comments, naming and programming style are clean and consistent;

Coursework Marking Guideline (60)

VERY GOOD SUBMISSION Mark (60)

all criteria of a good submission plus:

- + easily readable, fully effective & concisely commented code;
- + open ended work shows very good comprehension, and synthesis beyond just the examples taught;

The submitted material usually shows most of the following:

- + programming style and design adhere to the taught concepts and guidelines;
- + effective and efficient use of techniques;
- + demonstration of very good coding competence;
- + evidence of appropriate use of techniques throughout;
- + program design shows very good understanding of the underlying issues and concepts;

Coursework Marking Guideline (70)

EXCELLENT SUBMISSION Mark (70)

all criteria of a very good submission plus:

+ open ended work shows excellent comprehension, some originality, good code efficiency, and synthesis clearly beyond the bounds of taught approaches;

The submitted material usually shows most of the following:

- + programming style and design show professionalism and fully adhere to taught concepts and guidelines;
- + very effective and efficient use of techniques;
- + demonstration of excellent coding competence;
- + program design and structure reflect excellent understanding;
- + evidence of advanced ability, knowledge, creativity and effort

Coursework Marking Guideline (80)

MASTERY OF THE SUBJECT AREA Mark (80)

all criteria of an excellent submission plus:

- + open ended work shows mastery of advanced methods in all aspects;
- + truly impressive outcome showing advanced ability, creativity and outstanding effort throughout;

The submitted material usually shows most of the following:

- + highly professional programming style and design well beyond the taught concepts;
- + exceptional command of language features and techniques;
- + demonstration of outstanding coding competence;
- + program design and structure reflect brilliant understanding;

Coursework Marking Guideline (90-100)

NOVEL AND PUBLISHABLE Mark (90-100)

all previous criteria plus:

- + truly novel outcome, worthy of dissemination 'as is';
- + outcome with deep critical analysis and reflection;

The submitted material usually shows most of the following:

- + synthesis in an original way using ideas from across and/or beyond computer science;
- + extension of techniques beyond available language features creating novel capabilities;
- + work is of a standard usually published in a top research journal or conference;

The Exam

The exam is MUST-PASS and forms the core summative assessment with a weight of 80% of the unit mark. In it, we will test your ability to understand programs in Haskell and C and the principles behind them. The lectures, coursework, labs and worksheets all contribute towards building your understanding in order to master this exam.

The COMS10016 exam forms the first of two key MUST-PASS programming assessments in year one – the second one being a live viva in COMS10018 in the second teaching block.